

Roshan Singh

Generative AI Engineer | LLM, RAG & Agentic AI

roshan.16n@gmail.com · +91 85117 63382 · Pune, India

LinkedIn · GitHub · Portfolio · Open to relocation (Bengaluru / Hyderabad)

Summary

Generative AI Engineer with 2.5+ years at Bosch building and deploying production-grade LLM, RAG, and agentic AI applications in Python. Works across the applied-LLM stack – AI agents and tool integration (including MCP), RAG and retrieval, LLM evaluation, and guardrails – using LangChain, LangGraph, FastAPI, and Azure OpenAI. Owns features end to end, from design through deployment to getting them in front of real users.

Technical Skills

Core: Generative AI (GenAI), Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), AI Agents / Agentic AI, LLM Evaluation, Prompt Engineering, MLOps / LLMOps, Backend Engineering

Languages: Python, SQL, JavaScript

GenAI / LLM: RAG, GraphRAG, LangChain, LangGraph, MCP (Model Context Protocol), prompt engineering, AI agents (tool calling, multi-agent, ReAct), reranking, embeddings, semantic / hybrid search, RAG evaluation (RAGAS, recall@k), guardrails and Responsible AI (PII redaction)

LLMs / APIs: Azure OpenAI (GPT-4o), Anthropic Claude, Groq (Llama), Hugging Face Transformers

Vector / Data stores: Vector databases – FAISS, Pinecone, ChromaDB, Neo4j (graph); PostgreSQL, Redis

Backend / Cloud: FastAPI, REST APIs, microservices, async processing, Azure, Docker, Git, CI/CD, Linux

Frontend / Testing: React.js, Next.js, TypeScript, Streamlit, Playwright, LangSmith, pytest, uv

Work Experience

Bosch Global Software Technologies (BGSW)

Jan 2024 – Present

GenAI Engineer, Applied AI Team

Pune, India

- **CSAI Hub (org-wide AI app builder):** own the **UI agent** for an internal platform (in pilot, built for use across Bosch teams) that turns a plain-language idea into a working full-stack app – from interview to docs to code. My agent generates production React / Next.js + TypeScript UIs from the requirement, wires them to the generated backend APIs, and enforces the Bosch design system.
- Built the UI agent with LangGraph and Azure OpenAI / Claude, added a self-validation loop that generates, checks, and fixes the UI, and tests the output end to end with Playwright. Also built an **MCP server that serves Bosch design-system (ADUX) components** on demand – so the agent composes UIs from brand-compliant building blocks, the way a developer pulls in a component library.
- **Parts-data classification (first project; owned the build):** built the GenAI pipeline for a two-person PoC – embedding models (OpenAI text-embedding-3-large) with cosine-similarity matching and human-in-the-loop thresholds – to classify messy dealer-management-system parts data (inconsistent, multi-language, poorly categorized) into Bosch's official reporting structure, across four countries (~**2M records**). Hit the target categorization KPIs (70% sub-group, 20% position-group) on three of the four country datasets. Took part in the client meetings to understand requirements and walk through results.
- **DocupediaAI (enterprise RAG knowledge assistant):** built the RAG pipeline – document ingestion, chunking, embedding, and retrieval across enterprise data sources. Added Responsible-AI controls (input/output guardrails and PII redaction) so it stays compliant in regulated internal environments.
- **Agent-as-a-Service platform:** built ReAct and delegation-style agents on the team's internal agent platform, wrote MCP (Model Context Protocol) tool integrations connecting agents to internal systems, and contributed reusable orchestration components used by other teams.

- **MYA (multi-agent procurement platform):** designed and tuned the retrieval layer – the semantic-search and RAG components the agents depend on for grounded, citable answers.
- Shipped several internal GenAI POCs, including a cybersecurity RAG assistant over the Docupedia knowledge base (pulls context for vulnerability reports and drafts triage summaries) and an HR onboarding / talent-acquisition assistant for candidate screening.
- Recognized twice by Bosch for delivery speed and ownership (“Rockstar” and “Excellence” awards).

Projects

Multihop-GraphRAG

[GitHub](#) ↗

Hybrid graph + vector retrieval for multi-hop question answering

2025 – 2026

- Built a retrieval system that pairs dense vector search with Neo4j graph traversal, so it can follow relationships across multiple documents and answer multi-hop questions that single-chunk retrieval misses. An LLM-driven ETL step extracts entities and relationships into the graph.
- Fuses vector, keyword, and graph-traversal signals into one ranked result and returns answers with citations. Ships a retrieval inspector (shows why each result scored as it did), a graph explorer, a hybrid-vs-vector benchmark harness, and a live Streamlit demo in Docker.

Tech: Python, LangChain, Neo4j, vector search, hybrid retrieval, Groq, Streamlit, Docker

Agent Memory

[GitHub](#) ↗

Long-term memory for LLM agents – episodic / semantic / procedural

2026

- Built a memory layer that lets an LLM agent remember across sessions, with the three memory types from current research – episodic (what happened), semantic (durable facts and preferences), and procedural (learned how-to rules).
- Exposes the store / retrieve / update / summarize / discard operations as callable “memory-as-tools” the agent invokes itself; retrieval ranks by a blend of semantic similarity, salience, and recency. Runs without an API key via graceful degradation. 39 passing tests.

Tech: Python, LLM agents, tool calling, embeddings, semantic search, Groq, pytest, uv

Smart Retrieval Router

[GitHub](#) ↗

Reranking + cost-aware routing between small and frontier models

2026

- Built a retrieval pipeline with a second-stage reranker (lexical, cross-encoder, and LLM rerankers behind one interface) that reorders candidates so the most relevant passages land at the top before they reach the model.
- Adds a difficulty-based router that sends easy queries to a small model and hard ones to a frontier model, with a cost model that quantifies the savings vs. always using the frontier model for a given traffic mix. Reranking reorders candidates to lift the most relevant passages before the model. 30 passing tests.

Tech: Python, reranking, cross-encoders, model routing, cost optimization, nDCG, Groq

Education

Chandigarh University

2019 – 2023

B.E. Computer Science and Engineering CGPA: 7.88

Mohali, Punjab

Certifications

- Complete Generative AI with LangChain & Hugging Face – Udemy (Krish Naik), 2025. 57 hrs covering LangChain, RAG, vector databases, and LLM application development.